Universidad Rey Juan Carlos

Escuela / Facultad:

Campus:

TITULACIÓN:

ASIGNATURA:

D.N.I.:

APELLIDOS: García Delgado

NOMBRE: Natalia

FECHA:

CURSO:

GRUPO:

---

① (No puedo compilar)

a) $\vdash (p \vee q \rightarrow r) \rightarrow (r \rightarrow s) \rightarrow (p \rightarrow s) \wedge (q \rightarrow s)$ mayusc

```
def proof [P,Q,R,S] : (Either [P,Q] => R) => (R => S) =>
            (P => S) => (P=>S , Q=>S) =
    f1 => f2 => ( p => f2 (f1) (Left (p)),
            q => f2 (f1 (Right (q))
          )
```

minusc

b) $\vdash (p \rightarrow q) \vee (q \rightarrow p)$ Tercio

```
def proof [P,Q](em: Either [Q, NOT[Q]]):
    Either [P=>Q , Q=>P] =
      em match
          case Left (q) =>
              Left (p=>q)
          case Right (nq) =>
              Right (q=>nq(q))
```

Universidad Rey Juan Carlos

Escuela / Facultad:

Campus:

TITULACIÓN:

ASIGNATURA:

D.N.I.:

APELLIDOS: García Delgado NOMBRE: Natalia

FECHA: CURSO: GRUPO:

---

② A y B option [A] => B y (B, A => B)

a) option [A] => B

= 1+A => B

= B(1+A)

= B^1 * B^A

=

~~B+(A=>B)~~ ≡ (B, A => B)

B+(A=>B)

b)
```
class Iso [A,B] extends of Isomorphic (
  option [A] => B , (B,A => B) ) =
def FROM ( f: option [A] => B ) : (B, A => B) =
    f match
        case None         => ( )
        case option (a) => ( _ , a )

def to ( t: (B , A => B) ) : option [A] => B =
    t
```

③

a) Rec final

```
def collect [A, B] ( F : A => Option [B],
        list : List [A] ) : List [B] =

    list match
        case List() => List()
        case h::t =>
            F(h) match
          ✻ [ case Some(b) => ~~if b%2 == 0 then~~
                  b :: collect(F, t)
                else
              ~~case None~~ => collect(f, t)
```

```
✻ [ case Some(b) =>
        if b%2 == 0 then
          b :: collect(F,t)
        else
          collect(F,t)
    case None => collect
          (F,t)
```

b) FOLDRIGHT

```
def collect [A, B] (F : A => Option [B],
        list : List [A] ) : List [B] =
                                        acc ancc
    list.foldRight(List.Empty[B] ) (
        (a, acc) =>
            F(a) match
                           if b%2 == 0 ~~the~~ then
                case Some(b) => b::acc
                case None => acc
    )
            case Some(b) =>
                if b%2 == 0 then
                    b::acc
                else
                    acc
            case None => acc
```

Universidad Rey Juan Carlos

Escuela / Facultad:

Campus:

TITULACIÓN:

ASIGNATURA:

D.N.I.:

APELLIDOS: García Delgado NOMBRE: Natalia

FECHA: CURSO: GRUPO:

FLATMAP imap luego quita envoltorios

c)

```
def collect [A, B] (f: A => Option [B],
  list: List [A ) : List [B] =
  list.flatMap (a => f(a).toList())
```

~~④~~